Un pequeño consejo de seguridad/privacidad (Opcional pero de Matrícula)

Cuando hacemos un `include` del usuario completo, por defecto Sequelize envía **todo**, ¡incluyendo la contraseña encriptada (`password`) y el `token` del usuario! Para evitar filtrar datos sensibles, en los exámenes de DeliverUS solemos añadir un `attributes` dentro del include:

JavaScript

```javascript
include: [
  {
    model: User,
    as: 'customer',
    attributes: ['firstName'] // <- Así solo enviamos el nombre, y protegemos la
  }
]
```

1. `findOne()` (El Francotirador)

Busca **el primer registro** en la base de datos que cumpla las condiciones que le pongas en el `where`. En cuanto encuentra uno, para de buscar y te lo devuelve.

- **¿Qué devuelve?** Un `Objeto` (si lo encuentra) o `null` (si no existe).
- **¿Cuándo usarlo en un middleware?** Cuando la regla sea: *"Comprobar si existe ALGO específico"*.

```javascript
// Buscamos si ya existe UNA reseña de este usuario en este restaurant
const existingReview = await Review.findOne({
  where: {
    restaurantId: req.params.restaurantId, // El restaurante de la URL
    customerId: req.user.id                // El usuario del Token
  }
})
```

2. `count()` (El Contable)

Cuenta cuántas filas de la base de datos cumplen tus condiciones. La gran ventaja es que **no se trae los datos completos** (no descarga textos, fechas, etc.), solo le pide a la base de datos que le devuelva un número. Es rapidísimo.

- **¿Qué devuelve?** Un número `Integer` (0, 1, 5, 20...).
- **¿Cuándo usarlo en un middleware?** Cuando la regla hable de cantidades: *"Tiene que tener al menos un pedido", "No puede tener más de 3 direcciones guardadas"*.

```
// Contamos cuántos pedidos tiene este usuario en este restaurante
const pastOrdersCount = await Order.count({
  where: {
    restaurantId: req.params.restaurantId, // El restaurante de la URL
    userId: req.user.id                    // El usuario del Token
  }
})

// Si el contador es 0, es que nunca ha pedido aquí. ¡Bloqueado! (Error 409)
if (pastOrdersCount === 0) {
  return res.status(409).send('Debes realizar al menos un pedido en este restaurante para poder valorarlo.')
}
```

3. `findByPk()` (El Portero de Discoteca)

Significa *Find By Primary Key*. Busca exclusivamente por la columna `id`. Es la versión más rápida y especializada de `findOne`.

- **¿Qué devuelve?** Un `Objeto` o `null`.
- **¿Cuándo usarlo en un middleware?** Cuando te pasen un ID por la URL (ej. `/:restaurantId`) y necesites saber si ese elemento realmente existe antes de hacer nada con él. *(Nota: En DeliverUS, el middleware `checkEntityExists` ya hace esto por debajo por ti, pero es vital saber cómo funciona).*

```
const review = await Review.findByPk(req.params.reviewId)
if (review.customerId !== req.user.id) {
  return res.status(403).json({ message: 'You do not have permission to modify this review.' })
}
```

4. `findAll()` (El Camión de Mudanzas) - ⚠️ PELIGRO EN MIDDLEWARES

Busca y extrae **TODOS** los registros que cumplan las condiciones, con todos sus datos.

- **¿Qué devuelve?** Un `Array` de objetos. **OJO:** Si no encuentra nada, NO devuelve `null`, devuelve un array vacío `[]`.
- **¿Cuándo usarlo en un middleware? CASI NUNCA.** Si solo quieres saber si algo existe, usar `findAll` es matar moscas a cañonazos, porque satura la memoria del servidor trayendo datos que no vas a usar. Úsalo solo en los Controladores (ej. `index`) cuando realmente necesites enviar esa lista de datos al Frontend.

PASOS A SEGUIR PARA HACER UN CONTROLLER QUE ESTABLEZCA UN VALOR POR DEFECTO:

El proceso siempre tiene **dos pasos obligatorios**:

1. **Apagar a todos:** Buscar todos los elementos del usuario y ponerles `isDefault: false`.
2. **Encender al elegido:** Buscar el elemento específico que nos han pasado por la URL y ponerle `isDefault: true`.

```
const markDefault = async function (req, res) {
  try {
    // PASO 1: "Apagar" todas las direcciones de este usuario
    // Actualizamos TODAS las direcciones cuyo dueño sea el usuario actual
    await Address.update(
      { isDefault: false },
      { where: { userId: req.user.id } } // ¡Ojo! Solo las de este usuario, no las
    )

    // PASO 2: "Encender" solo la dirección que nos piden por la URL
    await Address.update(
      { isDefault: true },
      { where: { id: req.params.addressId } }
    )

    // PASO 3: Buscar la dirección actualizada para devolvérsela al Frontend
    const updatedAddress = await Address.findByPk(req.params.addressId)
```

1. ¿Por qué en `update` solo sale `isDefault` y no todos los campos?

La clave aquí está en leer detenidamente el **RF3** del examen. El enunciado dice: *"Como cliente, quiero marcar una dirección de envío como predeterminada..."*. +1

Fíjate que la ruta asociada a este requisito es un `PATCH /shippingaddresses/:shippingAddressId/default`.

- Un método `PUT` se usaría si quisiéramos editar toda la dirección entera (cambiar la calle, la ciudad, el código postal, etc.).
- Un método `PATCH` se usa cuando solo queremos modificar **un único detalle** de un elemento que ya existe.

2. ¿Por qué `isDefault` es `.optional()` en `create` si el diagrama dice que es obligatorio?

Esta es la mejor pregunta. Aquí estás viendo el choque entre dos mundos: **El Modelo de Base de Datos (UML)** y **El Payload de la Petición (API)**.

- **El Diagrama UML (Base de datos):** Tienes toda la razón. En el diagrama, `isDefault` no tiene un `[0..1]`, lo que significa que a nivel de base de datos es obligatorio (`allowNull: false`). La base de datos no aceptará que se guarde una fila sin saber si es `true` o `false`.
- **La Validación (API):** La validación define qué campos estamos **obligando al usuario (Frontend)** a enviarnos cuando rellena el formulario de "Nueva Dirección".

Si miras las pruebas de aceptación del **RF2**, hay una regla mágica: *"Si es la primera dirección del usuario, debe marcarse dicha dirección automáticamente como predeterminada"*.

```
const create = [
  check('startTime')
    .exists()
    .isString()
    .custom(validateTimeFormat),

  check('endTime')
    .exists()
    .isString()
    .custom(validateTimeFormat)
    .custom(validateEndTimeAfterStartTime) //
]
```

Si la entidad a crear tiene atributos tipo Time, en los Validation los tratamos como un String y luego se le aplica con .custom(fAux) la función auxiliar para parsear a Time.
Sin embargo, si fuera tipo Date JS si tiene métodos para ello: isDate(), .toDate()
En las migraciones y modelos, sí hay tipo de datos Time (type: Sequelize.TIME, type: DataTypes.TIME) y Date

DUDAS

1. Si en los RF no aparece error 404, habría que añadirlo o hay que ceñirse a lo que el enunciado diga:
   Habría que ponerlo.
2. En el examen de ShippingAddress, en el Validation, porqué no se pone la FK y en update sólo se pone el isDefault
   Mirar RF3: PATCH sirve para modificar un único detalle, en este caso, isDefault. Lo demás se queda como estaba
3. Cómo sé los controllers que tengo que poner en el controller de la entidad. Si te dicen delete > destroy, put > update, post > create… pero cómo sé que tengo que hacer un: `indexByRestaurant, show, indexProductsByCategory`
   Ceñirse a lo que digan los RF.
4. **¿¿Cuándo se ponen las FK en Validation??**
   Básicamente es una validación más, habría que fijarse y ver si ponerla o no. Pero las FK suelen haberse chequeado previamente. Sin embargo, ponerla otra vez indica un código más seguro.

🔴 **CUANDO ES OBLIGATORIO (El ID viaja en el Body)**

Debes poner la validación **cuando la ruta NO incluye el ID en la URL.**

Si tu archivo de rutas (`Routes.js`) dice esto:

`app.route('/products').post(...)`

- **¿Por qué es obligatorio?** Como la URL (`/products`) es genérica y no dice a qué restaurante pertenece el producto, el Frontend (la app móvil) está obligado a enviarte ese dato escondido dentro del paquete JSON (el `req.body`).

🟢 **CUANDO ESTÁ PROHIBIDO / ES REDUNDANTE (El ID viaja en la URL)**

NO debes poner la validación en `Validation.js` **cuando la ruta YA incluye el ID en la URL.**

Si tu archivo de rutas (`Routes.js`) dice esto:

`app.route('/restaurants/:restaurantId/products').post(...)`

- **¿Por qué no se pone?** Porque el ID ya viaja visible en la propia dirección web (`req.params`). El Frontend no te lo va a enviar en el JSON para no repetir datos.

5. Si tenemos en los middlewares ya hechas las validaciones (checkEntityExists, checkProductCategoryOwnership, checkNoProductsInCategory): al poner dichos middlewares en las routes, no haría falta poner en el controlador otra vez dichas validaciones, no??

Efectivamente

```
const destroy = async function (req, res) {
  try {
    const productCategory = await
ProductCategory.findByPk(req.params.productCategoryId)
    if (!productCategory) {
      return res.status(404).send('Product category not found')
    }

    await productCategory.destroy()
    res.status(204).send()
  } catch (err) {
    res.status(500).send(err)
  }
}
```

6. En el controller en create, hay que especificar las FK cogiéndolas de req para que el usuario no se las invente, no?: Sí

```
newProductCategory.userId = req.user.id // usuario actualmente autenticado
newProductCategory.restaurantId = req.params.restaurantId
```

7. Ver en la solución de ProductCategory la solución del controller y ver diferencia entre: index, indexByRestaurant, indexProductsByCategory

-> index: `SELECT * FROM ProductCategories;`

- No se usa en los routes, entonces para qué se crea? ?

-> indexByRestaurant: SELECT * FROM ProductCategories WHERE restaurantId = 5;

- Este sí es el que te piden directamente

-> indexProductsByCategory: todos los productos de una categoría concreta

- Este te lo piden de forma implícita (pues no tiene sentido mostrar las categorías de producto de un restaurante, y no enseñar los productos de cada categoría)

En definitiva, para qué se crea el index o hay que hacer los métodos CRUD en todos como esqueleto del controller, y si hiciera falta añadir alguno adicional pues se crearía?

Hay que ceñirse a lo que nos digan. En este caso nos pedían que listaramos las categorías de nuestro restaurante, por ende, sólo y digo sólo, habría que hacer el indexByRestaurant, los demás NO.

8. El orden de los middlewares en las rutas es relevante? Si es así, es el orden en el que aparecen en el RF?

Sí, no obstante, es algo subjetivo. Hay cosas lógicas como si te dicen *"Como propietario…"*, entonces habría que verificar primero que esté logueado y verificar el rol de *'owner'* antes de ver si existe la entidad o algo más.

9. Si te piden un PATCH como el siguiente, fijate en la ruta (en el Validation (update) sólo habría que poner un check para el atributo Default?):

Fíjate que la ruta asociada a este requisito es un `PATCH` `/shippingaddresses/:shippingAddressId/default`.

Sí, el PATCH solo se usa para actualizar unos valores específicos que te los indica el RF

10. Si te piden un PATCH, también habría que hacer el update en controller?

No, hay que ceñirse a lo que nos piden. En las soluciones de frontend viene el backend más completo de lo que nos piden a nosotros

11. check('userId').not().exists() >> comprobamos el usuario en los controllers mediante req.user.id para asegurarnos el usuario correcto

12. ¿Debo conocerme los errores HTTP?

No, con poner algún número de los que aparecen en el LAB2, o alguno parecido de algún middleware/test ya creado vale

13. Se pueden llevar apuntes hechos a mano??

NO, crear repositorio en github con documento y acordarse de la url

A la hora de hacer en controller el **create** hay varias formas de hacerlo, y todas son igual de válidas:

Opción 1: El atajo explícito (La que me has pasado)

JavaScript

```javascript
const review = await Review.create({
  stars: req.body.stars,
  body: req.body.body,
  restaurantId: req.params.restaurantId,
  customerId: req.user.id
})
```

- **Ventaja:** Es súper segura y fácil de leer. Le estás diciendo a Sequelize exactamente qué campos coger de cada sitio. Nadie te puede inyectar campos raros.
- **Desventaja:** Si tuvieras un modelo con 15 atributos (ej. un `User` con nombre, apellidos, teléfono, dirección, etc.), tendrías que escribir 15 líneas a mano.

Opción 2: Tu intuición (`build` + `save`)

Lo que tú proponías es perfectamente válido y es como se suele enseñar al principio porque ves los pasos más claros:

JavaScript

```javascript
// 1. Construimos el objeto (pero aún no está en la base de datos)
const newReview = Review.build({
  stars: req.body.stars,
  body: req.body.body
})

// 2. Le inyectamos las claves foráneas (FK) por seguridad
newReview.restaurantId = req.params.restaurantId
newReview.customerId = req.user.id

// 3. Ahora sí, lo guardamos en MySQL/Postgres
const review = await newReview.save()
```

- **Ventaja:** Es brutalmente clara. Separas lo que envía el usuario (estrellas y texto) de lo que inyecta tu sistema (IDs). Si antes de guardar tienes que hacer algún cálculo matemático (ej. calcular un impuesto), este es el mejor sitio para hacerlo.

Métodos auxiliares que crea Sequalize por si piden hacer alguna función auxiliar como avgStars() (reviews porque un restaurantes hasMany reviews):

Como el alias es `reviews` (en plural), Sequelize te regala por la cara todos estos métodos:

- `await this.getReviews()` -> Va a la BD y trae todas las valoraciones.
- `await this.countReviews()` -> Va a la BD y te dice cuántas hay (el número).
- `await this.hasReview(id)` -> Comprueba si tiene una valoración en concreto.
- `await this.addReview(review)` -> Le asigna una nueva valoración.

Si la relación fuera singular como con user (belongsTo):

Sequelize generaría métodos en singular:

- `await this.getUser()`
- `await this.setUser(user)`

Validación para ver si un Schedule pertenece a Restaurant:

```
const checkScheduleBelongsToRestaurantOnCreate = async (value, { req }) => {
  if (!value) return Promise.resolve() // Permitir scheduleId vacío

  const schedule = await Schedule.findByPk(value)
  if (!schedule) {
    return Promise.reject(new Error('The scheduleId does not exist.'))
  }
  if (schedule.restaurantId !== req.body.restaurantId) {
    return Promise.reject(new Error('The scheduleId does not belong to the given restaurantId.'))
  }
  return Promise.resolve()
}

const checkScheduleBelongsToRestaurantOnUpdate = async (value, { req }) => {
  if (!value) return Promise.resolve() // Permitir scheduleId vacío

  const schedule = await Schedule.findByPk(value)
  if (!schedule) {
    return Promise.reject(new Error('The scheduleId does not exist.'))
  }
  const product = await Product.findByPk(req.params.productId)
  if (product.restaurantId !== schedule.restaurantId) {
    return Promise.reject(new Error('The scheduleId does not belong to the restaurant of this product.'))
  }
  return Promise.resolve()
}
```

martes, 24 de marzo de 202

(No olvidar poner en el validation de create y update:
check('scheduleId').optional({ nullable: true, checkFalsy: true }).isInt({ min: 1 }).toInt().custom(checkScheduleBelongsToRestaurantOnCreate)
check('scheduleId').optional({ nullable: true, checkFalsy: true }).isInt({ min: 1 }).toInt().custom(checkScheduleBelongsToRestaurantOnUpdate)

)